Metafeature Selection via Multivariate Sparse-Group Lasso Learning for Automatic Hyperparameter Configuration Recommendation

Liping Deng, Wen-Sheng Chen, and Mingqing Xiao

***Abstract*— The performance of classification algorithms is mainly governed by the hyperparameter settings deployed in applications, and the search for desirable hyperparameter configurations usually is quite challenging due to the complexity of datasets. Metafeatures are a group of measures that characterize the underlying dataset from various aspects, and the corresponding recommendation algorithm fully relies on the appropriate selection of metafeatures. Metalearning (MtL), aiming to improve the learning algorithm itself, requires development in integrating features, models, and algorithm learning to accomplish its goal. In this article, we develop a multivariate sparse-group Lasso (SGLasso) model embedded with MtL capacity in recommending suitable configurations via learning. The main idea is to select the principal metafeatures by removing those redundant or irregular ones, promoting both efficiency and performance in the hyperparameter configuration recommendation. To be specific, we first extract the metafeatures and classification performance of a set of configurations from the collection of historical datasets, and then, a metaregression task is established through SGLasso to capture the main characteristics of the underlying relationship between metafeatures and historical performance. For a new dataset, the classification performance of configurations can be estimated through the selected metafeatures so that the configuration with the highest predictive performance in terms of the new dataset can be generated. Furthermore, a general MtL architecture combined with our model is developed. Extensive experiments are conducted on 136 UCI datasets, demonstrating the effectiveness of the proposed approach. The empirical results on the well-known SVM show that our model can effectively recommend suitable configurations and outperform the existing MtL-based methods and the well-known search-based algorithms, such as random search, Bayesian optimization, and Hyperband.**

***Index Terms*— Automatic hyperparameter recommendation, metafeature selection, metalearning (MtL), multivariate sparse-group Lasso (SGLasso).**

Manuscript received 8 September 2022; revised 14 January 2023; accepted 27 March 2023. Date of publication 10 April 2023; date of current version 4 September 2024. The work of Liping Deng and Mingqing Xiao was supported in part by the National Science Foundation-Division of Mathematical Sciences (NSF-DMS) of the United States under Grant 1854638. The work of Wen-Sheng Chen was supported in part by the Interdisciplinary Innovation Team of Shenzhen University and in part by the Natural Science Foundation of Shenzhen under Grant 20200815000520001. *(Corresponding author: Mingqing Xiao.)*

Liping Deng and Mingqing Xiao are with the School of Mathematical and Statistical Sciences, Southern Illinois University Carbondale, Carbondale, IL 62901 USA (e-mail: liping.deng@siu.edu; mxiao@siu.edu).

Wen-Sheng Chen is with the College of Mathematics and Statistics, Shenzhen University, Shenzhen, Guangdong 618060, China (e-mail: chenws@szu.edu.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TNNLS.2023.3263506, provided by the authors.

Digital Object Identifier 10.1109/TNNLS.2023.3263506

I. Introduction

CLASSIFICATION is one of the most important but challenging tasks in the study of machine learning [1], [2]. Hyperparameter tuning for classification algorithms is the critical step to generate the desirable classification performance and plays a crucial role in successful modeling.

During the last decade, many search-based approaches, such as random search (RS) [3], Hyperband [4], and Bayesian optimization (BO) [5], have been developed to seek the most-likely effective hyperparameter configurations (we call it configurations hereafter). However, these search methods are often computationally expensive, especially for large-scale problems, as a significant part of configuration space for every new dataset is generally required to evaluate from the scratch. Moreover, the default settings of classification algorithms in some libraries, e.g., `scikit-learn` and `MATLAB`, are directly adopted by nonexperts, but they often lead to either mediocre or even undesirable selections. Therefore, it is of practical importance to develop an efficient and effective automatic configuration recommendation approach that can provide more efficient hyperparameter selection in practices, particularly in various machine learning applications (see [6], [7]).

Metalearning (MtL) [7], or learning to learn, provides an alternative mechanism that allows us to select the promising configurations without new configuration evaluations (see [8], [9], [10], [11] and references therein). In its approach, the metadata, including metafeatures and classification performance of candidate configurations, is extracted from a collection of historical classification problems. Metafeatures are a group of measures that characterize datasets from various aspects, capturing the characteristics of an individual dataset uniquely [12], [13]. Accordingly, a *metalearner* is built on the metadata to identify the intrinsic relationship between metafeatures and historical performance, and therefore, the prospective configuration of a new problem can be inferred by its metafeatures. Because of their critical roles, both metalearners and metafeatures are the essential components for configuration recommendation tasks under the framework of MtL.

One of the representative metalearners is k-nearest neighbors algorithm (KNN) [8], which seeks the *similarity* between the new problem and the historical problems with the

2162-237X © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Universidade Federal de Alagoas. Downloaded on May 06,2026 at 12:56:00 UTC from IEEE Xplore. Restrictions apply.

consensus that the K most similar historical problems should share the same optimal configurations, and thus, the one that has the overall best performance among the K historical problems will be recommended to the new problem. Another common strategy is metaregression (see [14], [15], [16]), where metafeatures are predictors, while historical performance is viewed as the response, the performance of candidate configurations on the new dataset can, therefore, be predicted and ranked, and the one with highest predictive performance can be picked. Other strategies in MtL also include metaclassification, collaborative filtering (CF), and so on (see [17], [18], [19], [20], [21] and references therein). For these aforementioned approaches, metafeatures must be manually prescribed based on researchers' preferences or experience, and optimal effectiveness usually is not guaranteed.

Currently, there are mainly eight types of metafeatures in the literature, which are statistics and information theory, model structure (MS), (relative) landmarking (LM), structural information (SI), concept (CON), clustering (CLU), and problem complexity [12], [13]. Since metafeatures are the main source for the metalearner to generate the configuration recommendation, the study on the automatic selection of metafeatures appears to be practically attractive. Depending on the datasets, measures from different types of metafeatures are either redundant or correlated, and thus, metafeature selection often becomes necessary in order to achieve effective and efficient recommendations. However, to the best of our knowledge, this study in dealing with metafeature selection under the framework of MtL is quite limited. For example, Todorovski et al. [22] and Kalousis and Hilario [23] employed the wrapper method to conduct metafeature selection, where metalearner training is required for every round of metafeature subset search, which leads to a computationally expensive process. Similarly, Cruz et al. [24] adopted the binary particle swarm optimization to search the optimal metafeature subset, which is designed only for dynamic ensemble selection tasks and the computational issues that appeared in the wrapper method remain. Bilalli et al. [25] combined the principal component analysis with a partial correlation graph to discover the important latent features of metafeatures, but little time budget can be saved during the metafeature extraction phase.

In this article, we propose a novel multivariate sparse-group Lasso (SGLasso) method, aiming at automatic metafeature selection, for hyperparameter configuration recommendations under the framework of MtL. Different from the existing approaches, as an embedded feature selection model, SGLasso can inlay metafeature selection into the metalearner training process so that the desired metafeatures can be identified, and the optimal performance of the metalearner can be attained. More specifically, we first set up a multivariate group Lasso metaregression problem on a collection of historical datasets where metafeatures and historical classification performance of configurations are, respectively, applied as predictors and responses. Then, an efficient algorithm is developed to learn the model coefficients used for metafeature selection. For a new dataset, the classification performance of candidate configurations can be predicted by the metafeatures of the new dataset selected by the trained SGLasso model. As a result, the configuration with the highest predictive performance can be recommended for the new dataset. The main contributions of this article are summarized as follows.

1) Different from current approaches in the literature, we establish a new model via multivariate SGLasso (MSGL) learning, focusing on the metafeature selection through MtL. Technically, we adopt the $L_{2,1}$-norm (see [26], [27], [28]) in the regularization to reduce the number of model parameters and promote the sparsity between and within groups. An efficient learning algorithm via majorization minimization is developed for SGLasso training in the selection of principle metafutures.
2) The developed SGLasso model is embedded into the configuration recommendation under the framework of MtL, and accordingly, an automatic hyperparameter configuration architecture is fully developed and tested on 136 UCI datasets for applications.
3) Extensive experiments are conducted to demonstrate the effectiveness of our proposed approach. The empirical results have shown that our SGLasso is capable of recommending highly suitable configurations via feature selection and outperforming the existing MtL approaches and classical search algorithms, such as RS, BO, and Hyperband.

The structure of this article is given as follows. Some related studies about configuration recommendations under MtL are introduced in Section II. Our proposed SGLasso model is introduced in Section III, and its integration with MtL is developed in Section IV. The automatic configuration recommendation algorithm is developed in Section V. The empirical results and discussions are presented in Section VI. This article ends with concluding remarks in Section VII.

II. Metalearning-Based Recommendation

Currently, there are four common categories in the literature: similarity-based recommendation (SR), CF, classification, and regression, as illustrated next in detail.

A. Similarity-Based Recommendation

For a completely new classification task at hand, the similarity with the historical datasets can be characterized by defining adequate measurements between their metafeatures to determine the most suitable hyperparameters for the recommendation. KNN is usually adopted to select the K most similar datasets so that we can sort the configurations on each dataset and recommend the one that has the overall best performance to the new dataset [8], [29]. Soares et al. [8] first applied KNN to the kernel width selection for support-vector regression, in which the metadata on 42 datasets and 14 statistical metafeatures are generated. Similarly, Gomes et al. [29] built an MtL-based recommendation architecture on 40 regression problems to select both kernel width and regularization coefficient for SVM, and the suggested configuration by KNN is further fed to search algorithms, such as particle swarm optimization and Tabu search, to obtain more promising outcomes. In general, the number of neighbors K itself will

Authorized licensed use limited to: Universidade Federal de Alagoas. Downloaded on May 06,2026 at 12:56:00 UTC from IEEE Xplore. Restrictions apply.

affect the recommendation results. The KNN method only utilizes several of the closest datasets, and it does not make use of any information from other remaining historical datasets. Furthermore, KNN itself does not have the functionality for automatic metafeature selections.

B. Collaborative Filtering

Stern et al. [20] pioneered a new approach in MtL by utilizing the idea of model-based CF. The performance prediction of a configuration is treated as the estimation of a score that a user (dataset) may give to an item (configuration). Probabilistic matrix factorization (PMF) is employed to mine the latent variables of both items (configurations) and users (datasets). One issue of this approach is that one must evaluate some configurations in advance for a given new classification task. Yang et al. [30] employed a D-optimal experiment design to select the configuration evaluation subset that requires an extra computational effort. To deal with this issue, Misir and Sebag [21] trained a model, e.g., random forest, which maps metafeatures to the extracted variables to estimate the preference of the new dataset. However, this estimation often appears to be not accurate enough. Fusi et al. [31] leverage PMF and the Gaussian process to predict the performance of configurations, where Bayes optimization can be applied efficiently to search for the prospective configuration associated with a small number of iterations. The initialization of Bayes optimization is done by an SR, but the cold-starting issue remains, which leads to being less attractive.

C. Classification

Mantovani et al. [18] considered the configuration recommendation as a binary classification problem. They first evaluate SVM on 124 datasets using the default settings from `LibSVM` and `Weka` libraries and the best configurations acquired by search algorithms, and then, the datasets are divided into two classes: the one where default settings perform better is labeled as 1 and otherwise is labeled as 0. Six popular classification algorithms are, therefore, selected, e.g., decision tree (DT), naive Bayes, and multilayer perceptron, to fit the metadata accordingly. However, under this setting, it can only recommend two fixed configurations for new problems. Later, the same authors conducted a further study by focusing on the hyperparameter tuning of SVM within the same framework [19]. In other related literature, DTs are widely adopted as metalearners because of their ability in capturing the importance of metafeature variables [17], [32]. The notable drawback of classification with MtL is that some configurations may be biased by historical datasets, i.e., the configurations that are not included in labels will never be selected for new datasets.

D. Regression

To predict the performance of configurations, the MtL task can be formed as a regression problem. Gama and Brazdil [14] first utilized linear regression models to solve the problem of classification algorithm selection, where the classification errors are normalized so that they are comparable across different datasets. Sohn [15] built multivariate multiple regression models on 19 datasets where logistic transformation was applied to normalize the classification error rates. Brazdil et al. [16] and Bensusan and Kalousis [33] constructed the regression models using Cubist and kernel methods (KMs), where a larger pool of datasets and more types of metafeatures are considered, but their reasons for choosing these two regression models are not well justified, and the applicable scope is unclear. Later, support-vector regression with RBF kernel was adopted for nonlinear metaregression [34], [35]. Other models also include multilayer perceptron [36], [37] and self-organizing mapping [38]. Apparently, none of the aforementioned regression models considers metafeature selection, and they are mainly applied to fit the given metadata.

TABLE I

COMPARISON BETWEEN THE CHARACTERISTICS OF THE REPRESENTATIVE METALEARNERS FROM SR, CF, CLASSIFICATION, AND REGRESSION. THE NUMBERS 1, 2, 3, 4, AND 5 REPRESENT THE METALEARNER: UTILIZES ALL THE DATASETS, UTILIZES ALL CONFIGURATIONS, HAS THE MODEL INTERPRETABILITY, DOES NOT HAVE THE COLD-START PROBLEM, AND SELECTS METAFEATURE SUBSET, RESPECTIVELY

Meta-learner (Category)	1	2	3	4	5
KNN (SR)		✓		✓	
PMF (CF)	✓	✓	✓		
DT (Classification)	✓		✓	✓	
KM (Regression)	✓	✓		✓	
SGLasso (our model)	✓	✓	✓	✓	✓

In summary, we list the characteristics of the representative metalearners from each category, including our SGLasso model developed in this article, in Table I.

III. MULTIVARIATE SPARSE-GROUP LASSO

Before we introduce our proposed SGLasso model, we adopt the following common notations throughout this article. We use bold lowercase letters to denote vectors and bold uppercase letters to denote matrices. The identity (zero) matrix is written as $\mathbf{I}$ ($\mathbf{0}$) with appropriate dimensions. The entry located at the ith row and the jth column of a matrix $\mathbf{A}$ is represented by A_{ij}, and the ith entry of a vector $\mathbf{x}$ is denoted as x_i. We use $\mathbf{a}_i$ and $\mathbf{a}_{i\star}$ to represent the ith column and the ith row of matrix $\mathbf{A}$, respectively. The transpose of a matrix $\mathbf{A}$ (vector $\mathbf{x}$) is represented by $\mathbf{A}^\top$ ($\mathbf{x}^\top$). The trace of a square matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ is defined as $\mathrm{tr}(\mathbf{A}) := \sum_{i=1}^n A_{ii}$.

The Frobenius norm (L_2-norm) of a matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ (vector $\mathbf{x} \in \mathbb{R}^r$) is defined as $\|\mathbf{A}\|_F := (\sum_{i=1}^m \sum_{j=1}^n A_{ij}^2)^{1/2}$ ($\|\mathbf{x}\|_2 := (\sum_{i=1}^r x_i^2)^{1/2}$), and the L_1-norm of vectors is defined by $\|\mathbf{x}\|_1 := \sum_{i=1}^r |x_i|$. Especially, the $L_{2,1}$-norm of matrices used in this article is defined as

$$\|\mathbf{A}\|_{2,1} := \sum_{j=1}^n \sqrt{\sum_{i=1}^m A_{ij}^2} = \sum_{j=1}^n \|\mathbf{a}_j\|_2$$

where $\mathbf{a}_j$ is the jth column of matrix $\mathbf{A}$.

Authorized licensed use limited to: Universidade Federal de Alagoas. Downloaded on May 06,2026 at 12:56:00 UTC from IEEE Xplore. Restrictions apply.

A. Proposed Model

Suppose that $\mathbf{Y} = [\mathbf{y}_1, \mathbf{y}_2, \ldots, \mathbf{y}_n] \in \mathbb{R}^{m\times n}$ is the response matrix and each column $\mathbf{y}_i \in \mathbb{R}^m$ is an instance with continuous entries, and $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n] \in \mathbb{R}^{r\times n}$ is the design matrix and each column $\mathbf{x}_i \in \mathbb{R}^r$ is an instance. Then, a multivariate multiple linear regression model can be constructed as

$$\mathbf{Y} = \mathbf{W}\mathbf{X} + \mathbf{E} \tag{1}$$

where $\mathbf{E} = [\boldsymbol{\epsilon}_1, \boldsymbol{\epsilon}_2, \ldots, \boldsymbol{\epsilon}_n] \in \mathbb{R}^{m\times n}$ is the error term and $\boldsymbol{\epsilon}_i \sim \mathcal{N}_m(\mathbf{0}, \sigma_i^2\mathbf{I})$, $i = 1, 2, \ldots, n$, where $\mathcal{N}_m(\cdot,\cdot)$ stands for multivariate Gaussian distributions. Aiming at being able to conduct feature selection and capture the intrinsic connection between the predictors and responses, an MSGL model is constructed as

$$F(\mathbf{W}) = \frac{1}{2}\left\|\mathbf{Y} - \sum_{l=1}^{p}\mathbf{W}_l\mathbf{X}_l\right\|_F^2 + \lambda\sum_{l=1}^{p}\|\mathbf{W}_l\|_{2,1} \tag{2}$$

where $\mathbf{W} = [\mathbf{W}_1, \mathbf{W}_2, \ldots, \mathbf{W}_p] \in \mathbb{R}^{m\times r}$ is the coefficient matrix in which $\mathbf{W}_l \in \mathbb{R}^{m\times r_l}$, $l = 1, 2, \ldots, p$, and $\mathbf{X}_l \in \mathbb{R}^{r_l\times n}$ $(l = 1, 2, \ldots, p)$ with $\sum_{l=1}^{p} r_l = r$ are acquired by partitioning $\mathbf{X}$ rowwisely into p disjoint groups. $\lambda > 0$ is a tuning parameter that controls the sparsity of $\mathbf{W}$ in (2). Notice that model (2) degrades to multivariate Lasso (MLasso) [39] when $p = 1$.

To solve for $\mathbf{W}$ that minimizes (2), we adopt block-coordinate descent by fixing $\mathbf{W}_l, l \neq k$, and solve for $\mathbf{W}_k$ each time. Hence, we can rewrite the loss function of (2) by ignoring the terms that do not involve $\mathbf{W}_k$ as

$$F(\mathbf{W}_k) = \frac{1}{2}\|\mathbf{S}_{-k} - \mathbf{W}_k\mathbf{X}_k\|_F^2 + \lambda\|\mathbf{W}_k\|_{2,1} + C_{-k} \tag{3}$$

where $\mathbf{S}_{-k} = \mathbf{Y} - \sum_{l\neq k}^{p}\mathbf{W}_l\mathbf{X}_l$, and $C_{-k} = \lambda\sum_{l\neq k}^{p}\|\mathbf{W}_l\|_{2,1}$ is a constant. To simplify notations, we rewrite (3) as

$$F(\mathbf{B}) = \frac{1}{2}\|\mathbf{S} - \mathbf{B}\mathbf{H}\|_F^2 + \lambda\|\mathbf{B}\|_{2,1} + C \tag{4}$$

where $\mathbf{S} = \mathbf{S}_{-k} \in \mathbb{R}^{m\times n}$, $\mathbf{B} = \mathbf{W}_k \in \mathbb{R}^{m\times r_k}$, $\mathbf{H} = \mathbf{X}_k \in \mathbb{R}^{r_k\times n}$, and $C = C_{-k}$.

With some matrix algebras, the gradient of (4) with respect to $\mathbf{B}$ is given by

$$\nabla F(\mathbf{B}) = -\mathbf{S}\mathbf{H}^\top + \mathbf{B}\mathbf{H}\mathbf{H}^\top + \lambda\mathbf{U} \tag{5}$$

where $\mathbf{U} = [\mathbf{u}_1, \mathbf{u}_2, \ldots, \mathbf{u}_{r_k}]$ and

$$\mathbf{u}_i = \begin{cases} \dfrac{\boldsymbol{\beta}_i}{\|\boldsymbol{\beta}_i\|_2}, & \text{if } \|\boldsymbol{\beta}_i\|_2 \neq 0 \\ \mathbf{z}_i, & \text{if } \|\boldsymbol{\beta}_i\|_2 = 0 \end{cases} \tag{6}$$

where $\boldsymbol{\beta}_i$ is the ith columns of $\mathbf{B}$ and $\|\mathbf{z}_i\|_2 \leq 1$. Therefore, if $\|\boldsymbol{\beta}_i\|_2 \neq 0$, then $\boldsymbol{\beta}_i$ can be updated by letting $\nabla F(\mathbf{B}) = \mathbf{0}$ via

$$\boldsymbol{\beta}_i = \left(\mathbf{S}\mathbf{h}_{i\star}^\top - \sum_{j\neq i}^{r_k}\boldsymbol{\beta}_j\|\mathbf{h}_{j\star}\|_2^2\right)\left(\|\mathbf{h}_{i\star}\|_2^2 + \frac{\lambda}{\|\boldsymbol{\beta}_i\|_2}\right)^{-1}. \tag{7}$$

Otherwise, if $\|\boldsymbol{\beta}_i\|_2 = 0$, it satisfies

$$\left\|\mathbf{S}\mathbf{h}_{i\star}^\top - \sum_{j\neq i}^{r_k}\boldsymbol{\beta}_j\|\mathbf{h}_{j\star}\|_2^2\right\|_2 \leq \lambda. \tag{8}$$

In fact, updating rule (7) is unstable in the scenarios where $\|\mathbf{h}_{i\star}\|_2 \to 0$, e.g., the ith feature vector of $\mathbf{H}$ contains very small values, or it is extremely sparse. The analysis of the instability of (7) can be found in the Supplementary Material. Next, we tackle this issue via a majorization minimization approach.

B. Majorization Minimization

Denote

$$l(\mathbf{S}, \mathbf{B}) = \frac{1}{2}\|\mathbf{S} - \mathbf{B}\mathbf{H}\|_F^2 \tag{9}$$

and assume that $\mathbf{H}$ does not have zero rows, namely, $1/(\|\mathbf{h}_{j\star}\|_2^2)$, $j = 1, 2, \ldots, r_k$, are well-defined; then, we have the following theorem, and the detailed proof is given in the Supplementary Material for the sake of completeness.

Theorem 1: Centered at $\mathbf{B}^t$, a conformable constant matrix, (9) can be majorized as

$$l(\mathbf{S}, \mathbf{B}) \leq l(\mathbf{S}, \mathbf{B}^t) + \operatorname{tr}\Big((\mathbf{B} - \mathbf{B}^t)^\top\nabla l(\mathbf{S}, \mathbf{B}^t)\Big) + \frac{1}{2a}\|\mathbf{B} - \mathbf{B}^t\|_F^2 \tag{10}$$

where

$$0 < a \leq \min\left\{\frac{1}{\|\mathbf{h}_{1\star}\|_2^2}, \frac{1}{\|\mathbf{h}_{2\star}\|_2^2}, \ldots, \frac{1}{\|\mathbf{h}_{r_k\star}\|_2^2}\right\} \tag{11}$$

and

$$\nabla l(\mathbf{S}, \mathbf{B}^t) = -\mathbf{S}\mathbf{H}^\top + \mathbf{B}^t\mathbf{H}\mathbf{H}^\top.$$

By adding $\lambda\|\mathbf{B}\|_{2,1} + C$ to the both sides of (10), one has

$$F(\mathbf{B}) \leq G(\mathbf{B}, \mathbf{B}^t) \tag{12}$$

where

$$G(\mathbf{B}, \mathbf{B}^t) = l(\mathbf{S}, \mathbf{B}^t) + \operatorname{tr}\Big((\mathbf{B} - \mathbf{B}^t)^\top\nabla l(\mathbf{S}, \mathbf{B}^t)\Big) + \frac{1}{2a}\|\mathbf{B} - \mathbf{B}^t\|_F^2 + \lambda\|\mathbf{B}\|_{2,1} + C. \tag{13}$$

It is easy to see that minimizing (13) is equivalent to minimizing (4). Without loss of generality, taking the derivative of (13) with respect to $\boldsymbol{\beta}_i$, i.e., the ith column of $\mathbf{B}$, one has

$$\nabla G(\boldsymbol{\beta}_i, \boldsymbol{\beta}_i^t) = \frac{1}{a}(\boldsymbol{\beta}_i - \tilde{\boldsymbol{\beta}}_i^t) + \lambda\mathbf{u}_i \tag{14}$$

where $\tilde{\boldsymbol{\beta}}_i^t = \boldsymbol{\beta}_i^t - a\nabla l(\mathbf{S}, \boldsymbol{\beta}_i^t)$ and $\mathbf{u}_i$ is given in (6). Letting $\nabla G(\boldsymbol{\beta}_i, \boldsymbol{\beta}_i^t) = \mathbf{0}$ and with some algebras, we have $\boldsymbol{\beta}_i = \mathbf{0}$ if

$$\|\tilde{\boldsymbol{\beta}}_i^t\|_2 \leq a\lambda \tag{15}$$

and otherwise

$$\boldsymbol{\beta}_i = \tilde{\boldsymbol{\beta}}_i^t\left(1 + \frac{a\lambda}{\|\boldsymbol{\beta}_i\|_2}\right)^{-1}. \tag{16}$$

Taking L_2-norm on both sides of (16) and combine the result of (15), one obtains

$$\|\boldsymbol{\beta}_i\|_2 = \left(\|\tilde{\boldsymbol{\beta}}_i^t\|_2 - a\lambda\right)_+ \tag{17}$$

Authorized licensed use limited to: Universidade Federal de Alagoas. Downloaded on May 06,2026 at 12:56:00 UTC from IEEE Xplore. Restrictions apply.

where $(\cdot)_+$ is an operator that maps negative entries to zeros. Plugging (17) into (16), one can have

$$\begin{aligned}\boldsymbol{\beta}_i &= \tilde{\boldsymbol{\beta}}_i^t\left[1+\frac{a\lambda}{\left(\|\tilde{\boldsymbol{\beta}}_i^t\|_2 - a\lambda\right)_+}\right]^{-1}\\ &= \tilde{\boldsymbol{\beta}}_i^t\left(1-\frac{a\lambda}{\|\tilde{\boldsymbol{\beta}}_i^t\|_2}\right)_+.\end{aligned} \tag{18}$$

The uniqueness solution of $\boldsymbol{\beta}_i$ in (16) is shown in the Supplementary Material.

Proposition 1: Equation (18) is the unique solution of (16).

Now, we combine the updating rules with all columns of $\mathbf{B}$, which can be expressed as

$$\mathbf{B} = \tilde{\mathbf{B}}^t(\mathbf{I} - a\lambda\boldsymbol{\Sigma})_+ \tag{19}$$

where $\tilde{\mathbf{B}}^t = \mathbf{B}^t - a\nabla l(\mathbf{S}, \mathbf{B}^t)$ and

$$\boldsymbol{\Sigma} = \begin{pmatrix} \frac{1}{\|\tilde{\boldsymbol{\beta}}_1^t\|_2} & \cdots & 0 \\ \vdots & \vdots & \vdots \\ 0 & \cdots & \frac{1}{\|\tilde{\boldsymbol{\beta}}_n^t\|_2}\end{pmatrix}$$

is a diagonal matrix. The updating rule (19) can be computed efficiently since $(\mathbf{I} - a\lambda\boldsymbol{\Sigma})_+$ is a diagonal matrix, and some diagonal entries may be zero.

C. Some Related Existing Group Lasso Approaches

Group Lasso was initially developed by Yuan and Lin [40] for the univariate regression problems; the proposed objective function is given by

$$\min_{\boldsymbol{\beta}_1,\boldsymbol{\beta}_2,\ldots,\boldsymbol{\beta}_p}\left\|\mathbf{y} - \sum_{l=1}^{p}\mathbf{X}_l^\top\boldsymbol{\beta}_l\right\|_2^2 + \lambda\sum_{l=1}^{p}\sqrt{g_l}\|\boldsymbol{\beta}_l\|_2$$

where $\mathbf{y} \in \mathbb{R}^n$, $(g_l)^{1/2}$ depends on the group sizes, and $\boldsymbol{\beta}_l \in \mathbb{R}^{r_l}, i = 1, 2, \ldots, p$, and $\sum_{l=1}^{p} r_l = r$. This model is allowed to drop some groups of variables, i.e., to impose the between-group sparsity by setting the corresponding parameter vector $\boldsymbol{\beta}_l$ to be zeros via tuning parameter λ. To add sparsity within the groups, Simon et al. [41] proposed a SGLasso, which is formulated as

$$\min_{\boldsymbol{\beta}_1,\boldsymbol{\beta}_2,\ldots,\boldsymbol{\beta}_p}\left\|\mathbf{y} - \sum_{l=1}^{p}\mathbf{X}_l^\top\boldsymbol{\beta}_l\|_2^2 + \lambda_1\sum_{l=1}^{p}\|\boldsymbol{\beta}_l\|_2 + \lambda_2\sum_{l=1}^{p}\|\boldsymbol{\beta}_l\right\|_1.$$

This model can enforce both within and between group sparsity via a traditional L_1-norm constraint under the univariate regression setting. To deal with high-dimensional cases, an MSGL appeared in [42] is given by

$$\min_{\mathbf{W}\in\mathbb{R}^{m\times r}}\left\|\mathbf{Y} - \sum_{l=1}^{p}\mathbf{W}_l\mathbf{X}_l\right\|_F^2 + \lambda\sum_{l=1}^{p}\|\mathbf{W}_l\|_F + \lambda_G\sum_{j,k}|W_{jk}|$$

where $W_{jk} \in \mathbf{W}$ and $\mathbf{W} = [\mathbf{W}_1, \mathbf{W}_2, \ldots, \mathbf{W}_p]$. This approach considers the elementwise sparsity of $\mathbf{W}$, and $\mathbf{W}$ can be rowwisely sparse; thus, the predicted response $\mathbf{y}$ for any given predictor $\mathbf{x}$ must be sparse too, which can present undesirable restrictions for linear regression tasks. Moreover, such an approach is not suitable for the metafeature selection addressed in this article since sparsity or small value of features may lead to numerical instability during the optimization process. We will address this issue under our approach setting.

IV. Metalearning via SGLasso

In this section, we apply our proposed SGLasso model to MtL for automatic configuration recommendations. Suppose that there are m configurations in the search space $\boldsymbol{\Omega}_{\mathcal{A}}$ of the investigated algorithm $\mathcal{A}$, namely, $\boldsymbol{\Omega}_{\mathcal{A}} = \{\boldsymbol{\omega}_1, \boldsymbol{\omega}_2, \ldots, \boldsymbol{\omega}_m\}$, and then, an m-dimensional classification performance vector on a historical dataset $\mathcal{D}$, denoted by $\mathbf{y} = [y_1, y_2, \ldots, y_m]^\top$, can be obtained, in which each entry y_i represents the classification performance of $\mathcal{A}$ instantiated configuration $\boldsymbol{\omega}_i$. Let f be a metafeature extraction function, and then, $\mathbf{x} = [x_1, x_2, \ldots, x_r]^\top = f(\mathcal{D})$ is the extracted metafeature vector of $\mathcal{D}$. How to collect metadata will be explained in Section VI-A. Now, suppose that there are n historical datasets $\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_n$, and then, a response matrix $\mathbf{Y} = [\mathbf{y}_1, \mathbf{y}_2, \ldots, \mathbf{y}_n]$ and a metafeature matrix $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n]$ can be obtained, where $\{\mathbf{x}_i, \mathbf{y}_i\}$ is the metadata of dataset $\mathcal{D}_i$.

A. SGLasso Training

The training procedures of SGLasso are summarized in Algorithm 1, where the superscript j refers to the jth iteration. To show the alternative updating process, we adopt the notations used in formulas (2) and (3). The group index vector $\boldsymbol{\gamma}$ has the same dimension as $\mathbf{x}$, i.e., $\boldsymbol{\gamma} \in \mathbb{R}^r$, and each entry of $\boldsymbol{\gamma}$ specifies which group the metafeature belongs to. In real scenarios, we can either generate $\boldsymbol{\gamma}$ randomly or determine it based on the natural grouping properties of metafeatures. In step 4, we use $a_l = (1/10)\min\{1/(\|\mathbf{x}_{1\star}^l\|_2^2), 1/(\|\mathbf{x}_{2\star}^l\|_2^2), \ldots, 1/(\|\mathbf{x}_{r_l\star}^l\|_2^2)\}, l = 1, 2, \ldots, p$, for each group based on the local steepest descent along with the objective function, where $\mathbf{x}_{j\star}^l$ is the jth row of $\mathbf{X}_l$. In step 10, we apply the Nesterov momentum to accelerate the convergence process.

Within the inner loop, i.e., from step 7 to step 10, the most computationally expensive part is the calculation of the gradient in step 7. Recall that

$$\begin{aligned}\nabla l(\mathbf{S}_{-l}, \mathbf{W}_l) &= -\mathbf{S}_{-l}\mathbf{X}_l^\top + \mathbf{W}_l\mathbf{X}_l\mathbf{X}_l^\top\\ &= (-\mathbf{S}_{-l} + \mathbf{W}_l\mathbf{X}_l)\mathbf{X}_l^\top\\ &= (-\mathbf{Y} + \mathbf{W}\mathbf{X})\mathbf{X}_l^\top\end{aligned} \tag{20}$$

which has the computational complexity of $O(mnr)$ where m, n, and r are the number of candidate configurations, the number of historical datasets, and the number of measures, respectively. In fact, $\mathbf{W}$ usually contains zero columns, which reduces the computation complexity to $O(mnr')$, where $r' < r$ is the number of nonzero columns of $\mathbf{W}$.

B. Illustrating Example

We here present an illustrating but simple example to show the effectiveness of our proposed SGLasso approach. Suppose that there are four historical problems, and we extract

Authorized licensed use limited to: Universidade Federal de Alagoas. Downloaded on May 06,2026 at 12:56:00 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 SGLasso($\mathbf{Y}, \mathbf{X}, \lambda, p, \boldsymbol{\gamma}$, maxL)

Input: Response matrix $\mathbf{Y}$, meta-features $\mathbf{X}$, tuning parameter λ, number of groups p, group index $\boldsymbol{\gamma}$, maximal allowable number of iterations maxL

Output: Estimated effects: $\mathbf{W} = [\mathbf{W}_1, \mathbf{W}_2, \ldots, \mathbf{W}_p]$

Categorize $\mathbf{X}$ into p groups $\{\mathbf{X}_1, \mathbf{X}_2, \ldots, \mathbf{X}_p\}$ using $\boldsymbol{\gamma}$
Initialize $\mathbf{W}^0 = [\mathbf{W}_1^0, \mathbf{W}_2^0, \ldots, \mathbf{W}_p^0]$, where $\mathbf{W}_l^0 \neq \mathbf{0}$, $l = 1, 2, \ldots, p$
Initialize $\boldsymbol{\Theta}_l^0 = \mathbf{W}_l^0, l = 1, 2, \ldots, p$
Calculate $a_l, l = 1, 2, \ldots, p$
for $j = 1 :$ maxL **do**
  for $l = 1 : p$ **do**
    compute gradient: $\mathbf{G}_l^{j-1} = \nabla l(\mathbf{S}_{-l}, \mathbf{W}_l^{j-1})$
    update $\tilde{\mathbf{W}}_l^j \leftarrow \mathbf{W}_l^{j-1} - a_l \mathbf{G}_l^{j-1}$
    update $\boldsymbol{\Theta}_l^j$ by $\boldsymbol{\Theta}_l^j \leftarrow \tilde{\mathbf{W}}_l^j (\mathbf{I} - a_l \lambda \boldsymbol{\Sigma})_+$
    update the center via a Nesterov step by $\mathbf{W}_l^j \leftarrow \boldsymbol{\Theta}_l^j + \frac{j}{j+3}(\boldsymbol{\Theta}_l^j - \boldsymbol{\Theta}_l^{j-1})$

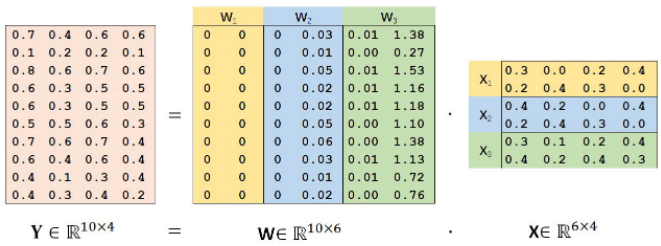


Fig. 1. Illustrating example that shows the training process of SGLasso. $\mathbf{Y}$, $\mathbf{X}$ and $\mathbf{W}$ represent the historical performance, metafeatures, and estimated outcome, respectively.

the historical performance $\mathbf{Y} \in \mathbb{R}^{10\times 4}$ and the metafeatures $\mathbf{X} \in \mathbb{R}^{6\times 4}$, respectively (see Fig. 1). We partition the six metafeatures into three groups, namely, $\boldsymbol{\gamma} = [1, 1, 2, 2, 3, 3]$ [see Fig. 1 (right)], where each color indicates a group of $\mathbf{X}$. Then, we compute $a_l, l = 1, 2, 3$, and we obtain $a_1 = 0.2990$, $a_2 = 0.2898$, and $a_3 = 0.2257$. We set $\lambda = 0.4$ and maxL$=$ 200. When the training is completed under our proposed algorithm, we obtain the estimated $\mathbf{W}$ (in the middle of Fig. 1). One can observe that $\mathbf{W}_1$ is a zero matrix, and the first column of $\mathbf{W}_2$ is also zero, while $\mathbf{W}_3$ is dense, which indicates that the first three metafeatures are assumed to be redundant and, thus, will be removed from the model. By taking a closer look into $\mathbf{X}$, we can see that the first and third rows of $\mathbf{X}$ are highly correlated with the fifth and sixth rows, respectively, and the second and the fourth rows are exactly the same. Thus, the proposed SGLasso successfully identifies the correlated information of metafeatures and provides the desirable solution.

C. Metafeature Selection

Our proposed SGLasso model can control both between-group and within-group sparsities by appropriately setting the tuning parameter λ. After the maxL iterations, the coefficient matrix $\mathbf{W} = [\mathbf{W}_1, \mathbf{W}_2, \ldots, \mathbf{W}_p]$ can be obtained, which contains zero columns as expected. Let $\mathbf{v} \in \mathbb{R}^{r'}$ be a vector containing the indices of the nonzero columns of $\mathbf{W}$ where $r' < r$. Then, $\mathbf{v}$ contains the indexes of the selected metafeatures. The relationship between the historical performance and the selected metafeatures over the n historical datasets can be rewritten as

$$\mathbf{Y} = \mathbf{W}_{[:,\mathbf{v}]}\mathbf{X}_{[\mathbf{v},:]} \tag{21}$$

where $\mathbf{W}_{[:,\mathbf{v}]}$ ($\mathbf{X}_{[\mathbf{v},:]}$) represents the submatrix that acquired by taking the columns (rows) of $\mathbf{W}$ ($\mathbf{X}$) indexed by $\mathbf{v}$. Three advantages of metafeature selection are ready to see: first, it reduces the number of the predictors, so the complexity of the model can be largely reduced [43]; second, by removing those irrelevant or redundant metafeatures for a specific problem, the model accuracy has been enhanced [44], [45]; and third, we only need to extract the selected metafeatures from a new dataset, yielding the efficiency improvement for the online phase.

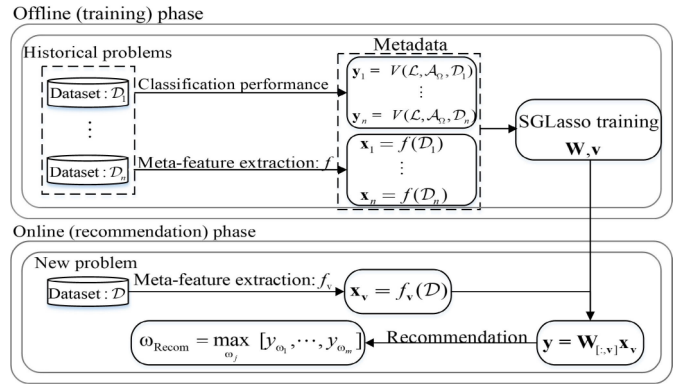


Fig. 2. Automatic configuration recommendation architecture based on SGLasso model, where $V(\cdot, \cdot, \cdot)$ is the adopted tenfold stratified cross-validation, and $\mathcal{L}$ is the balanced classification accuracy.

D. Performance Prediction

For a new dataset $\mathcal{D}$ of interest, we can estimate the performance of algorithm $\mathcal{A}$ over the configuration space $\boldsymbol{\Omega}_{\mathcal{A}}$ via the trained SGLasso model. We are first to extract the metafeatures of $\mathcal{D}$, denoted as $\mathbf{x}_{\mathbf{v}} = f_{\mathbf{v}}(\mathcal{D})$, where $\mathbf{x}_{\mathbf{v}} \in \mathbb{R}^{r'}$, and $f_{\mathbf{v}}$ means to extract the metafeatures indexed by $\mathbf{v}$ only. Then, the estimated performance is given by

$$\mathbf{y}_{\mathcal{D}} = \mathbf{W}_{[:,\mathbf{v}]}\mathbf{x}_{\mathbf{v}} \tag{22}$$

where $\mathbf{y}_{\mathcal{D}} \in \mathbb{R}^m$ and $(y_{\mathcal{D}})_j$, $j = 1, 2, \ldots, m$, is the predictive performance of $\boldsymbol{\omega}_j$ in the search space $\boldsymbol{\Omega}_{\mathcal{A}}$.

V. Automatic Configuration Recommendation

The description of the proposed SGLasso-based automatic configuration recommendation architecture is shown in Fig. 2. This architecture consists of two phases: offline training and online recommendation, respectively.

A. Offline Phase

The important ingredients of the offline phase are metadata extraction on a collection of historical datasets and the SGLasso training. We next explain how to collect the metadata under our approach.

Authorized licensed use limited to: Universidade Federal de Alagoas. Downloaded on May 06,2026 at 12:56:00 UTC from IEEE Xplore. Restrictions apply.

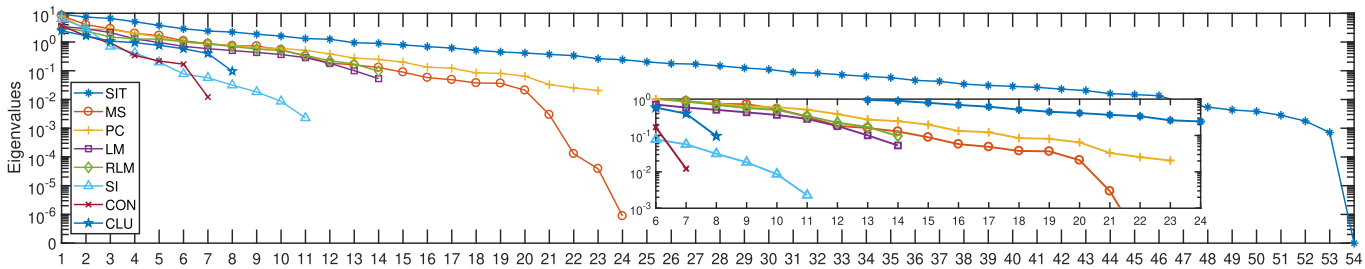


Fig. 3. Problem of multicollinearity existed in the eight types of metafeatures. Their eigenvalues being close to 0 imply that certain variables of metafeatures are highly correlated.

1) Classification Performance Evaluation: Suppose that there are n historical datasets $\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_n$ and m configurations, namely, $\mathbf{\Omega}_{\mathcal{A}} = \{\boldsymbol{\omega}_1, \boldsymbol{\omega}_2, \ldots, \boldsymbol{\omega}_m\}$. Then, the historical performance matrix $\mathbf{Y} = [\mathbf{y}_1, \mathbf{y}_2, \ldots, \mathbf{y}_n] \in \mathbb{R}^{m \times n}$ can be obtained using tenfold cross-validation, in which the stratified dataset splitting is considered to eliminate the influence of class imbalance that commonly existed in the real datasets. For this reason, the employed classification performance metric is described by the balanced classification accuracy.

2) Metafeature Extraction: The extraction of metafeatures involves the calculations of the measures defined in each metafeature. There are eight types of metafeatures that are widely used for MtL in practices [12], [13]. These metafeatures focus on different aspects, such as statistics & information theory (SIT), LM, relative LM (RLM), MS, SI, classification complexity (PC), CLU, and CON. The metafeature used by our SGLasso is the union of the eight metafeatures. In our approach, Python package `pymfe` [46] is used to extract these metafeatures, and then, several preprocessings are applied to the raw metafeatures: 1) every measure is normalized into [0, 1] using min–max scaling; 2) some measures that contain only one value throughout all datasets or contain too many missing values are deleted; and 3) we delete those repetitive measures that have the exactly the same values, and only one is kept. Therefore, the number of measures of each type of metafeature adopted in the experiments is reduced, as shown in the parentheses in Table II.

Here, we study the multicollinearity existing in the aforementioned metafeatures. Following the detection methods suggested by Mansfield and Helms [47], we first extract each type of metafeatures from 136 UCI datasets and obtain 136 metafeature vectors accordingly, and then, we compute their correlation matrix separately after the metafeature preprocessing and calculate the eigenvalues of each correlation matrix. We display the results in Fig. 3, from which one can see that some eigenvalues of the eight metafeatures are quite small (close to 0), which indicates that certain variables of the metafeatures are highly correlated. Therefore, the linear regression model without a careful metafeature selection usually will not generate desirable performance due to the ill-conditioned problem that occurred.

B. Recommendation

We have shown that the performance of $\mathcal{A}$ on a new dataset $\mathcal{D}$ can be estimated by (22). Then, the promising configurations can be recommended based on the order of the predictive performance. Detailedly, we sort $\mathbf{y}_{\mathcal{D}} = [y_1, y_2, \ldots, y_m]^\top$ in descending order as

$$\left[y_{i_1}, y_{i_2}, \ldots, y_{i_m}\right]$$

where $y_{i_1} \geq y_{i_2} \geq \cdots \geq y_{i_m}$. Therefore, the configuration that has the highest predictive value is recommended for this new dataset $\mathcal{D}$, i.e.,

$$\boldsymbol{\omega}_{\text{Recom}} = \mathbf{\Omega}_{\mathcal{A}}\{i_1\} \tag{23}$$

where $\mathbf{\Omega}_{\mathcal{A}}\{i_1\}$ stands for the (i_1)th element of the configuration space $\mathbf{\Omega}_{\mathcal{A}}$.

TABLE II

EXISTING METAFEATURES AND THE NUMBER OF MEASURES

Names	#measures (adopted)
Statistics&Information-theory (SIT)	73 (54)
Model structure (MS)	24 (24)
Problem complexity (PC)	35 (23)
Landmaking (LM)	14 (14)
Relative landmarking (RLM)	14 (14)
Structural information (SI)	14 (11)
Concept (CON)	8 (7)
Clustering (CLU)	8 (8)
Total	190 (155)

VI. EXPERIMENTS

In this section, we validate the recommendation effectiveness of our proposed model. We first present our experimental settings, including adopted datasets, classification algorithm, configuration space, and various comparative methods with suitable evaluation metrics, then present the empirical results, and provide further elaboration.

A. Experimental Setup

1) Datasets: We collect 136 classification datasets from UCI machine learning repository [48], in which the smallest dataset contains 88 instances and the maximal sample size is about 30k; the number of attributes ranges from 3 to 1558, and the number of classes ranges from 2 to 15. The statistical information in terms of the number of instances, classes, and dimensions of the adopted datasets can be found in the Supplementary Material. In experiments, 60, 20, and 56 datasets are randomly chosen as the training set, the validation set, and the testing set, respectively.

Authorized licensed use limited to: Universidade Federal de Alagoas. Downloaded on May 06,2026 at 12:56:00 UTC from IEEE Xplore. Restrictions apply.

2) Classification Algorithm and Configuration Space: We evaluate our proposed configuration recommendation algorithm on the well-known SVM [49] due to its excellent classification performance shown in many real applications, while its configuration evaluations, especially for large-scale datasets, are usually quite time-consuming.

Here, we consider SVM by using the RBF kernel because of the flexibility of its kernel parameter setting. There are two numerical hyperparameters required to be determined: the regularization coefficient `C` and the RBF kernel width `gamma`. Following the guideline provided in [50], we preset `C` $\in 2^{-5:1:15}$ and `gamma` $\in 2^{-15:1:3}$. These two settings are effective for many datasets. Therefore, there are $21 \times 19 = 399$ points in total in the configuration space of SVM. In the experiment, we adopt the SVM software developed in `scikit-learn` library.

3) Comparative Baselines: Three common MtL baselines are evaluated: 1) KNN [8], where Euclidean distance is used as the similarity measurement; 2) kernel KNN (KKNN) [16], [33], and the performance estimation of a dataset $\mathcal{D}$ is given by $\mathbf{y}_{\mathcal{D}} = (1/S)\sum_{j=1}^{n} k(\mathbf{x}_j, \mathbf{x}_{\mathcal{D}})\mathbf{y}_j$, where $S = \sum_{j=1}^{n} k(\mathbf{x}_j, \mathbf{x}_{\mathcal{D}})$, and $k(\cdot,\cdot)$ is RBF kernel; and 3) back-propagation neural network (BPNN) [36], [37]. The network structure is r-100-200-399. We select sigmoid as the activation function with the L_2 regularizer being added to avoid overfitting. We also compare our approach with MLasso, the special case of (2) when $p = 1$, and MSGL [42] to show the superiority of our feature selection model. All comparative approaches adopt the recommendation strategy proposed in (23).

On the other hand, we compare our method with four search-based algorithms: RS, BO, heteroscedastic and evolutionary BO (HEBO) [51], and Hyperband. During the configuration search process, tenfold cross-validation is used to evaluate each set of configurations selected by search algorithms. Besides, the performance of the default setting used by `scikit-learn` is also given to evaluate our approach.

4) Evaluation Metrics: Three commonly used metrics in MtL are used for evaluating the performance of the recommendation approaches, namely, (average) classification accuracy rate (CA, ACA), (average) recommendation accuracy rate (RA, ARA), and hit rate (HR). Their definitions can be found in [11]. The ranges of all metrics are [0, 1], and a larger value means that the recommended configuration is closer to the most desirable one, and thus, it is preferred.

B. Experimental Reports and Elaborations

Our experimental reports consist of three parts. The first part shows the ability in the metafeature selection of the proposed SGLasso, and the second and third parts compare the recommendation capacity between SGLasso and other MtL baselines, as well as four classical search algorithms. The best parameters for the SGLasso and the comparative MtL baselines are determined by handout validation, i.e., to evaluate the validation set on the model built on the training set and candidate parameters, the parameter that has the highest total performance, i.e., ACA + ARA + HR, is leveraged, and the results of the testing set evaluated on the best model are reported.

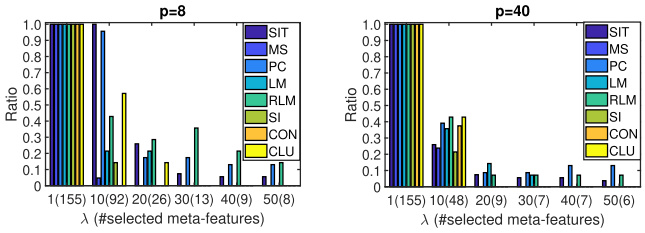


Fig. 4. Importance of metafeatures when different λ values are applied. The numbers of the selected metafeatures are given in the parenthesizes on the horizontal axis. A ratio being 0 means that the corresponding metafeature is completely removed from the model.

1) Effectiveness of Metafeature Selection: This section studies the SGLasso model based on three important questions. First, which metafeatures are essential for model construction? Second, can the problem of multicollinearity be reduced by metafeature selection? Third, how the recommendation performance of SGLasso will be affected by metafeature selection?

To show the importance of each type of metafeature for the investigated metamodel, the tuning parameter $\lambda = \{1, 10, 20, 30, 40, 50\}$ is set as an increasing sequence. As we discussed, a larger λ leads to a higher degree of sparsity of $\mathbf{W}$, and thus, fewer measures would be retained. To train the SGLasso model, we set $p = 40$, i.e., the eight types of metafeatures are randomly partitioned into 40 groups, and `maxL` $= 300$. Here, $p = 40$ is determined by the grid search, and it produces the best recommendation performance, as shown in the following subsection. The results are summarized in Fig. 4, where every data are the ratio between the number of the retained measures and the number of the total measures for an individual metafeature. The number of the selected metafeatures for each λ is given in the parenthesis below the horizontal axis. For the reason of comparison, we also provide results when $p = 8$ where each type of metafeature is a group to see whether grouping strategy will affect the model construction of SGLasso.

One can see that, for $p = 40$, no measures are deleted when $\lambda = 1$, but there are only 48 of them left when $\lambda = 10$, the amounts are reducing sharply when $\lambda \geq 20$, and there are only six measures when $\lambda = 50$. One can also observe that SIT, PC, LM, and RLM seem to provide the most important predictor variables for our metaregression problem since others are totally removed from the model as λ increases. We can draw similar conclusions for $p = 8$, which indicates that the grouping strategy applied to the SGLasso model does not affect the conclusion about which type of metafeature mainly describes the model. This is a fine property that evidences the model's reliability.

The second discussion is to see if the variable selection of SGLasso can reduce the multicollinearity as we showed previously. To save space, we only show the six smallest eigenvalues of the correlation matrix obtained from the selected metafeatures for each λ in Fig. 5. Again, we show the results of both $p = 40$ and $p = 8$. As expected, variables are severely correlated when $\lambda = 1$ as many eigenvalues are close to zero; however, this situation is alleviated as a smaller proportion of measures is selected as λ increases, which means that our

Authorized licensed use limited to: Universidade Federal de Alagoas. Downloaded on May 06,2026 at 12:56:00 UTC from IEEE Xplore. Restrictions apply.

TABLE III
PARAMETER SETTINGS FOR SGLASSO, MLASSO, MSGL, KNN, KKNN, AND BPNN

Approach	Parameter	Usage	Range	References
SGLasso	p	number of groups	$\{2,3,\cdots,50\}$	
	λ	sparsity regularization coefficient	$\{1,10,20,30,40,50\}$	–
MLasso	λ	sparsity regularization coefficient	$\{1,10,20,30,40,50\}$	[39]
MSGL	p	number of groups	$\{2,3,\cdots,50\}$	
	λ	sparsity regularization coefficient	$\{1,2,\cdots,10,20,\cdots,100\}\times 10^{-4}$	[42]
	λ_G	L_2 regularization coefficient	$\{1,2,\cdots,10,20,\cdots,100\}\times 10^{-4}$	
KNN	K	number of neighbors	$\{1,2,\cdots,50\}$	[8]
KKNN	γ	parameter of RBF kernel	$\{1e-3,1e-2,1e-1,1e0,1,2,3,4,5,1e1,2e1,5e1,1e2\}$	[16], [33]
BPNN	λ	L_2 regularization coefficient	$\{1e-7,1e-6,1e-5,1e-4,1e-3,1e-2\}$	[36], [37]

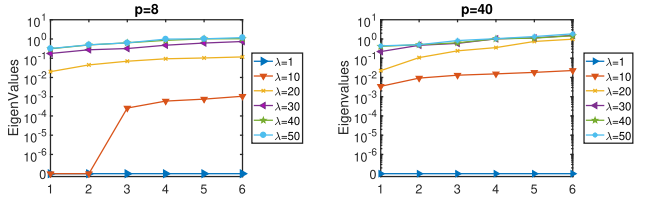


Fig. 5. Six smallest eigenvalues of the correlation matrix generated by the selected metafeature for each λ. Eigenvalues that are close to zero indicate a very high degree of multicollinearity existed in the metafeature.

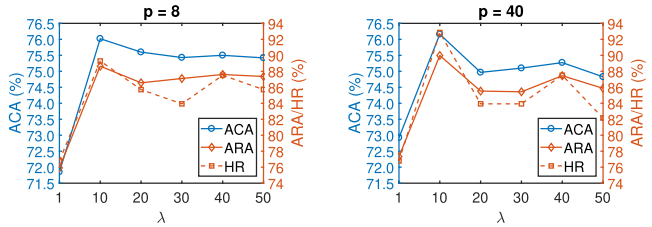


Fig. 6. Recommendation performance fluctuation of SGLasso when different degrees of sparsity are applied.

TABLE IV
COMPARISONS ON ACA, ARA, AND HR BETWEEN MLASSO, MSGL, KNN, KKNN, BPNN, AND SGLASSO. THE MAXIMAL VALUE OF EACH ROW IS HIGHLIGHTED

Metrics	SGLasso	MLasso	MSGL	KNN	KKNN	BPNN
ACA	**76.16**	74.77	75.07	74.74	75.11	74.70
ARA	**89.97**	85.95	87.22	85.54	87.00	85.31
HR	**92.86**	85.71	89.89	80.36	85.71	85.18

TABLE V
WILCOXON SIGNED-RANK TESTS BETWEEN MLASSO, MSGL, KNN, KKNN, BPNN, AND SGLASSO. HERE ">" REPRESENTS "IS BETTER THAN." P-VALUES THAT ARE LARGER THAN 0.05 ARE UNDERLINED

Alternative hypothesis	win/tie/lose	p-value (CA)	p-value (RA)
SGLasso>MLasso	36/4/16	0.002	0.013
SGLasso>MSGL	31/5/20	<u>0.088</u>	0.000
SGLasso>KNN	31/3/22	<u>0.052</u>	0.024
SGLasso>KKNN	35/3/18	0.007	0.025
SGLasso>BPNN	38/0/18	0.001	0.018

SGLasso can effectively delete those redundant and correlated measures and, thus, improve the model performance. In addition, the tendencies of $p = 40$ and $p = 8$ shown in Fig. 5 are consistent, which again proves the model stability of SGLasso. On the other hand, by comparing Fig. 3 ($\lambda = 1$) and Fig. 5, we can see that the combined metafeatures are more severely correlated than every single type of metafeature because more eigenvalues are zero (we show only six of them, but the total number is 20), which implies that there are similar measures cross different metafeatures and feature selection serves to remove those redundancies as expected.

Next, we show how the recommendation performance with our SGLasso demonstrates as the metafeature selection is performed. The results of ACA, ARA, and HR are depicted in Fig. 6, from which one can observe that feature selection indeed improves the recommendation performance: when all measures are applied ($\lambda = 1$), SGLasso has the mediocre performance with an ACA of 72.92%, an ARA of 77.31%, and an HR of 76.79% for $p = 40$, and an ACA of 71.83%, an ARA of 75.92%, and an HR of 76.79% for $p = 8$; however, it has the smallest ACA (ARA, HR) of 74.83% (85.43%, 82.14%) after the feature selection for $p = 40$ and has the smallest ACA (ARA, HR) of 75.43% (86.56%, 83.92%) after the feature selection for $p = 8$. On the other hand, the optimal recommendation occurs when $\lambda = 10$ for both grouping strategies, where ACA (ARA, HR) mounts to 76.16% (89.97%, 92.86%) for $p = 40$ and 76.02% (88.64%, 89.29%) for $p = 8$. As a comparison, the ACA of the real optimal configurations on the testing problems is 78.70%.

2) Comparisons With MtL Baselines: This section discusses the recommendation capacity of SGLasso compared to other MtL baselines, such as KNN, KKNN, BPNN, MSGL, and MLasso. The parameter search space of each approach is listed in Table III. The outcomes of ACA, ARA, and HR are summarized in Table IV, and the comparisons on every single dataset are shown in Fig. 7. Notice that the comparative baselines (KNN, KKNN, and BPNN) use the complete metafeatures, i.e., no feature selection is imposed as SGLasso does.

From Table IV, one can observe the following.

1) MtL-based recommendation can choose the applicable configurations since the achieved lowest ACA, ARA, and HR are 74.70% (BPNN), 85.31% (BPNN), and 80.36% (KNN), respectively, which proves the promising ability of MtL for automatic configuration selection.
2) Our method, SGLasso, outperforms other approaches with an ACA of 76.16%, an ARA of 89.97%, and an HR of 92.86%. MSGL wins the second place, and KKNN, MLasso, and BPNN take the tertiary, fourth, and fifth places, respectively, while KNN performs the worst.

Authorized licensed use limited to: Universidade Federal de Alagoas. Downloaded on May 06,2026 at 12:56:00 UTC from IEEE Xplore. Restrictions apply.

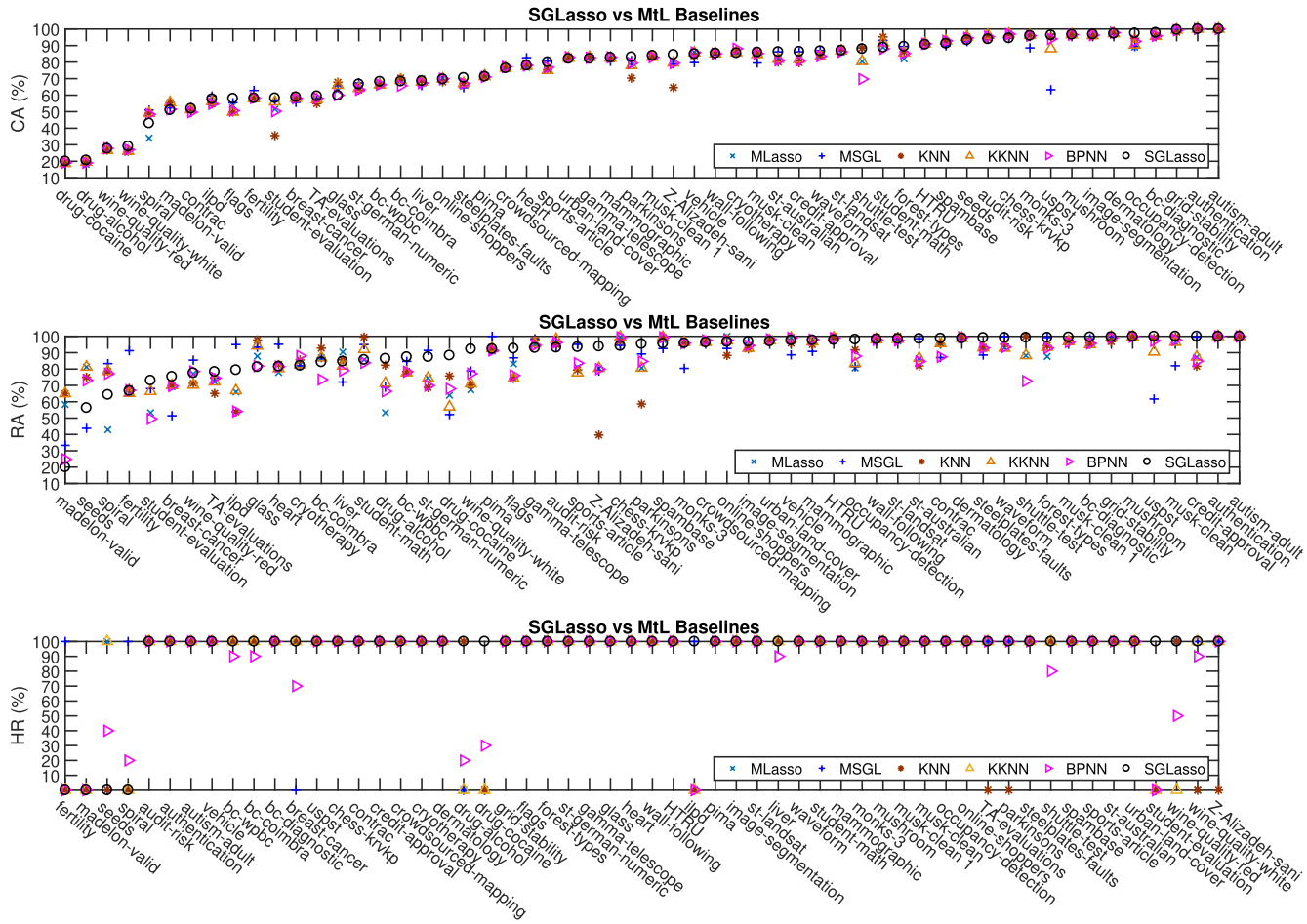


Fig. 7. Comparisons on CA (top), RA (middle), and HR (bottom) between MLasso, MSGL, KNN, KKNN, BPNN, and SGLasso on each single testing dataset.

3) SGLasso outweighs MLasso with a large gap, which implies that the importance of the grouping strategy is added to the model.

By looking into the performance difference on each dataset (see Fig. 7), one can see that SGLasso overpasses or performs equally with other baselines on most of the datasets, and this conclusion is more evident in terms of metric RA and HR, which testifies that CA alone is not enough to evaluate the recommendation performance. To be specific, SGLasso has an RA that is larger than 80% on 48 over 56 cases and has an RA that is larger than 90% on 37 over 56 cases, and these statistics are 42/28, 45/32, 39/32, 41/30, and 37/27 for MLasso, MSGL, KNN, KKNN, and BPNN, respectively; our method hits on 52 over 56 cases, and MLasso, MSGL, KNN, KKNN, and BPNN, respectively, hit on 48, 50, 45, 48, and 47 out of 56 cases. All approaches fail to hit on the dataset `madelon-valid`, and only MSGL hits on datasets `fertility` and `spiral`.

In addition, to show if SGLasso performs statistically better than other methods, we adopt Wilcoxon signed-rank tests at a 0.05 significant level. The null hypothesis is that SGLasso performs worse than or equally with MLasso (or MSGL, KNN, KKNN, and BPNN). We test this on both CA and RA, and the testing results are displayed in Table V, where the p-values that are less than 0.05 mean that the null hypothesis is rejected, and we accept the alternative hypothesis, that is, SGLasso outweighs significantly. We also give the statistics about how many datasets SGLasso wins, ties, and loses. One can see that SGLasso is significantly better than MLasso, KKNN, and BPNN on both CA and RA, and better than MSGL and KNN on RA. Although SGLasso fails to overpass MSGL and KNN on CA statistically, the p-values are close to 0.05. It is worth mentioning that KNN has a distinct manner in Tables IV and V, i.e., KNN has the worst HR, but it has high p-values. This is due to KNN having an unstable performance, i.e., it performs well on some datasets, but it does not generate satisfying recommendations on other datasets because of the difficulty of the selection of parameter `K`.

3) Comparisons With Search Algorithms: In this section, we compare the effectiveness between SGLasso and search algorithms when the same time budget is allowed. The maximal time budget provided for search algorithms on each dataset is the time consumed by SGLasso for online applications, i.e., metafeature extraction and performance estimation. Here, we exclude the time of offline training since this can be done whenever needed. Notice that we set $\lambda = 10$ and $p = 40$, so only 48 out of 155 measures are extracted during the online phase. In a contrast, KNN is also compared to these

Authorized licensed use limited to: Universidade Federal de Alagoas. Downloaded on May 06,2026 at 12:56:00 UTC from IEEE Xplore. Restrictions apply.

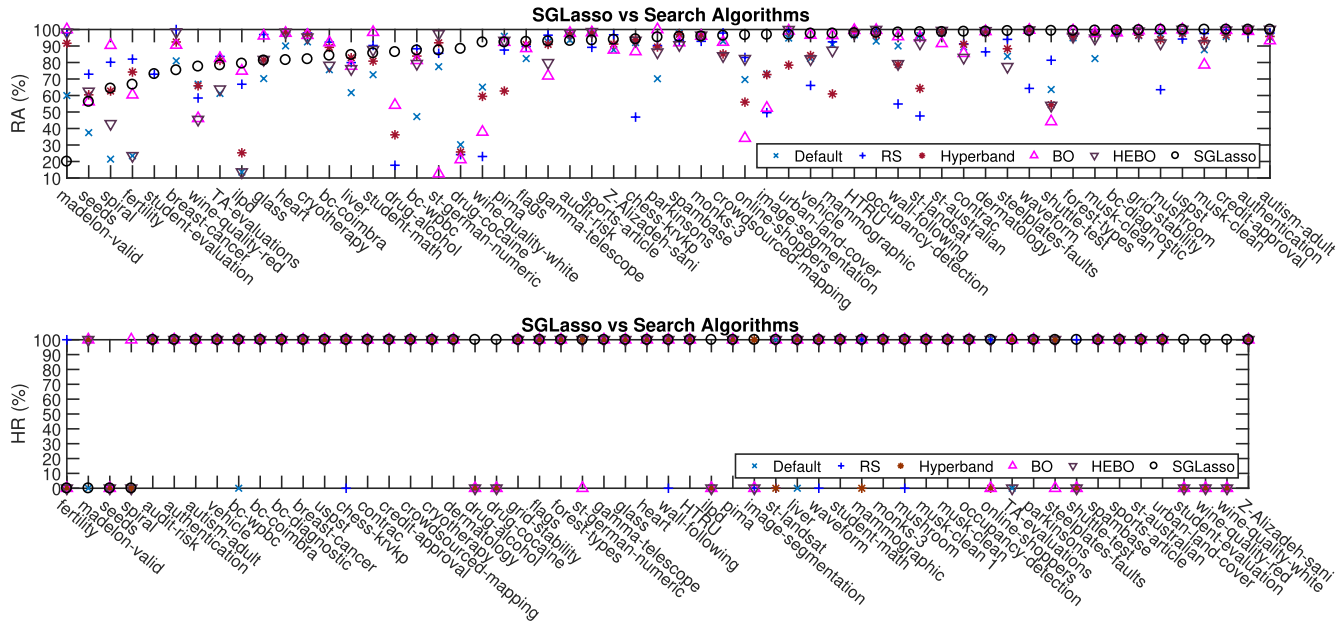


Fig. 8. Comparisons on RA (top) and HR (bottom) between default, RS, Hyperband, BO, HEBO, and SGLasso on each single testing dataset.

TABLE VI

COMPARISONS ON ACA, ARA, AND HR BETWEEN DEFAULT, RS, HYPERBAND, BO, HEBO, AND SGLASSO (KNN). THE MAXIMAL VALUE OF EACH ROW IS HIGHLIGHTED

Metrics	default	RS	Hyperband	BO	HEBO	SGLasso	RS	Hyperband	BO	HEBO	KNN
ACA	72.59	73.19	73.12	73.31	73.09	**76.16**	72.26	74.99	**75.95**	74.60	74.74
ARA	77.09	82.76	81.52	80.78	78.15	**89.97**	79.66	86.37	**89.57**	82.96	85.54
HR	75.00	75.00	76.79	76.79	78.57	**92.86**	80.36	80.36	**89.29**	82.14	80.36

TABLE VII

WILCOXON SIGNED-RANK TESTS BETWEEN DEFAULT, RS, HYPERBAND, BO, HEBO, AND SGLASSO (KNN). HERE, ">" REPRESENTS "IS BETTER THAN." P-VALUES THAT ARE LARGER THAN 0.05 ARE UNDERLINED

Hypothesis	win/tie/lose	p-value (CA)	p-value (RA)	Hypothesis	win/tie/lose	p-value (CA)	p-value (RA)
SGLasso>default	45/1/10	0.000	0.000	KNN>default	39/1/16	0.003	0.002
SGLasso>RS	37/0/19	0.028	0.026	KNN>RS	24/1/31	0.694	0.706
SGLasso>Hyperband	39/0/17	0.003	0.001	KNN>Hyperband	25/1/30	0.436	0.460
SGLasso>BO	35/4/17	0.026	0.021	KNN>BO	19/4/33	0.965	0.965
SGLasso>HEBO	34/2/20	0.002	0.003	KNN>HEBO	25/3/28	0.821	0.787

search algorithms where KNN needs more time budget to extract the whole 155 measures. We select KNN because it has larger p-values in the statistical tests, as given in Table V. The comparative results are summarized in Table VI, and the details on each dataset are depicted in Figs. 8 and 9. Because of the randomness, we repeat RS and Hyperband three times, and the average performance is reported.

From Table VI, one can see that: 1) the existing default configuration does not produce satisfying performance with an ACA of 72.59%, an ARA 77.09%, and an HR of 75.00%, which verifies the necessity of hyperparameter tuning; 2) RS, Hyperband, BO, and HEBO have the similar performance that is slightly better than the default; and 3) there is no doubt that SGLasso outweighs both default setting and search algorithms with a wide gap. Looking into the per-dataset comparisons (see Fig. 8), we find that there are some cases where search algorithms surpass SGLasso, e.g., `madelon-valid`, `breast-cancer`, and `heart`. These datasets have small dimensions, i.e., the small number of instances (attributes and classes), in which search algorithms are certainly more effective. As expected, SGLasso shows the superiority on those large-scale datasets; see `shuttle-test`, `online-shopper`, `drug-cocaine`, and so on, where cross-validation is time-consuming during each search. We also report the Wilcoxon signed-rank tests among the investigated approaches in Table VII, from which we can conclude that the superiority of our method over default, RS, Hyperband, BO, and HEBO is statistically significant.

However, the conclusions regarding KNN are different to some degree. Since the allowable time budget is increasing on each dataset, the performance of search algorithms has been improved, especially for BO, HEBO, and Hyperband. In Table VI, KNN outperforms default and RS, and it is inferior to Hyperband slightly, but BO outweighs KNN with a noticeable difference. The statistical tests in Table VII also show that KNN fails to beat search algorithms, and BO is even significantly better than KNN. In Fig. 9, it is clear to see that KNN is inferior to search algorithms on many datasets. These conclusions suggest that MtL without

Authorized licensed use limited to: Universidade Federal de Alagoas. Downloaded on May 06,2026 at 12:56:00 UTC from IEEE Xplore. Restrictions apply.

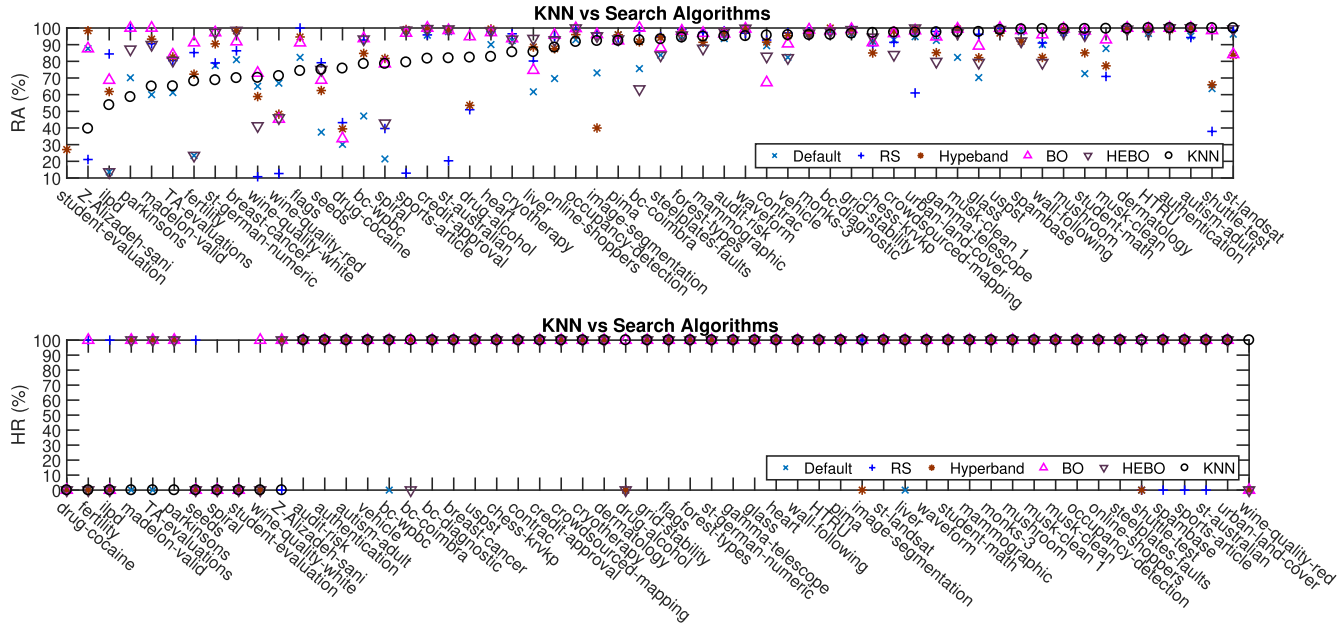


Fig. 9. Comparisons on RA (top) and HR (bottom) between default, RS, Hyperband, BO, HEBO, and KNN on each single testing dataset.

metafeature selection is not efficient when the time-consuming extraction of metafeatures is involved.

VII. Conclusion

Effective MtL mainly depends on the quality of metafeatures, and thus, the metafeature selection appears to be critical for algorithm learning under the framework of machine learning. In this article, by integrating features and algorithm learning as a whole, we develop a Lasso-type model, SGLasso, for the feature selection with the application to the automatic configuration recommendation. The main goal is to capture the intrinsic relationship between the dataset characteristics and the empirical performance through the development of a metalearner, which can identify key metafeatures and remove redundant or irrelevant ones. In the experiments, eight types of state-of-the-art metafeatures are adopted by our SGLasso to recommend configurations for SVM. The proposed SGLasso has shown superior performance that outperforms the existing MtL-based methods and the well-known search-based algorithms, such as RS, BO, and Hyperband. In addition, our approach is flexible and is ready to apply to other similar multivariate multiple regression problems where feature selection is involved with MtL. Our future study will further develop a nonlinear version of our SGLasso, and such outcomes will be reported elsewhere.

References

[1] S. Zhang, X. Li, M. Zong, X. Zhu, and R. Wang, "Efficient kNN classification with different numbers of nearest neighbors," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 5, pp. 1774–1785, May 2018.

[2] Z. Ghafoori, S. M. Erfani, S. Rajasegarar, J. C. Bezdek, S. Karunasekera, and C. Leckie, "Efficient unsupervised parameter estimation for one-class support vector machines," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 10, pp. 5057–5070, Oct. 2018.

[3] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *J. Mach. Learn. Res.*, vol. 13, pp. 281–305, Feb. 2012.

[4] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, "Hyperband: A novel bandit-based approach to hyperparameter optimization," *J. Mach. Learn. Res.*, vol. 18, no. 1, pp. 6765–6816, 2017.

[5] J. Snoek, H. Larochelle, and R. P. Adams, "Practical Bayesian optimization of machine learning algorithms," in *Proc. Adv. Neural Inf. Process. Syst.*, 2012, pp. 2951–2959.

[6] R. Leite, P. Brazdil, and J. Vanschoren, "Selecting classification algorithms with active testing," in *Proc. Int. Workshop Mach. Learn. Data Mining Pattern Recognit.* Berlin, Germany: Springer, 2012, pp. 117–131.

[7] J. Vanschoren, "Meta-learning," in *Automated Machine Learning*. Cham, Switzerland: Springer, 2019, pp. 35–61.

[8] C. Soares, P. B. Brazdil, and P. Kuba, "A meta-learning method to select the kernel width in support vector regression," *Mach. Learn.*, vol. 54, no. 3, pp. 195–209, Mar. 2004.

[9] C. Soares and P. B. Brazdil, "Selecting parameters of SVM using meta-learning and kernel matrix-based meta-features," in *Proc. ACM Symp. Appl. Comput.*, Apr. 2006, pp. 564–568.

[10] S. Ali and K. A. Smith-Miles, "A meta-learning approach to automatic kernel selection for support vector machines," *Neurocomputing*, vol. 70, nos. 1–3, pp. 173–186, 2006.

[11] L. Deng and M. Xiao, "Latent feature learning via autoencoder training for automatic classification configuration recommendation," *Knowl.-Based Syst.*, vol. 261, Feb. 2023, Art. no. 110218.

[12] I. Khan, X. Zhang, M. Rehman, and R. Ali, "A literature survey and empirical study of meta-learning for classifier selection," *IEEE Access*, vol. 8, pp. 10262–10281, 2020.

[13] P. Brazdil, J. N. van Rijn, C. Soares, and J. Vanschoren, "Dataset characteristics (metafeatures)," in *Metalearning*. Berlin, Germany: Springer, 2022, pp. 53–75.

[14] J. Gama and P. Brazdil, "Characterization of classification algorithms," in *Proc. Portuguese Conf. Artif. Intell.* Berlin, Germany: Springer, 1995, pp. 189–200.

[15] S. Y. Sohn, "Meta analysis of classification algorithms for pattern recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 21, no. 11, pp. 1137–1144, Nov. 1999.

[16] P. Brazdil, J. N. V. Rijn, C. Soares, and J. Vanschoren, "Metalearning approaches for algorithm selection II," in *Metalearning*. Berlin, Germany: Springer, 2022, pp. 77–102.

[17] S. Ali and K. A. Smith, "On learning algorithm selection for classification," *Appl. Soft Comput.*, vol. 6, no. 2, pp. 119–138, Jan. 2006.

[18] R. G. Mantovani, A. L. Rossi, J. Vanschoren, and A. C. de Carvalho, "Meta-learning recommendation of default hyper-parameter values for SVMs in classification tasks," in *Proc. MetaSel@ PKDD/ECML*, 2015, pp. 80–92.

[19] R. G. Mantovani, A. L. D. Rossi, J. Vanschoren, B. Bischl, and A. C. P. L. F. Carvalho, "To tune or not to tune: Recommending when to adjust SVM hyper-parameters via meta-learning," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2015, pp. 1–8.

Authorized licensed use limited to: Universidade Federal de Alagoas. Downloaded on May 06,2026 at 12:56:00 UTC from IEEE Xplore. Restrictions apply.

[20] D. Stern, R. Herbrich, T. Graepel, H. Samulowitz, L. Pulina, and A. Tacchella, "Collaborative expert portfolio management," in *Proc. 24th AAAI Conf. Artif. Intell. (AAAI)*, 2010, pp. 179–184.

[21] M. Misir and M. Sebag, "Alors: An algorithm recommender system," *Artif. Intell.*, vol. 244, pp. 291–314, Mar. 2017.

[22] L. Todorovski, P. Brazdil, and C. Soares, "Report on the experiments with feature selection in meta-level learning," in *Proc. PKDD Workshop Data Mining, Decis. Support, Meta-Learn. ILP: Forum Practical Problem Presentation Prospective Solutions*. Citeseer, 2000, pp. 27–39.

[23] A. Kalousis and M. Hilario, "Feature selection for meta-learning," in *Proc. Pacific-Asia Conf. Knowl. Discovery Data Mining*. Berlin, Germany: Springer, 2001, pp. 222–233.

[24] R. M. O. Cruz, R. Sabourin, and G. D. C. Cavalcanti, "META-DES. Oracle: Meta-learning and feature selection for dynamic ensemble selection," *Inf. Fusion*, vol. 38, pp. 84–103, Nov. 2017.

[25] B. Bilalli, A. Abelló, and T. Aluja-Banet, "On the predictive power of meta-features in OpenML," *Int. J. Appl. Math. Comput. Sci.*, vol. 27, no. 4, pp. 697–712, Dec. 2017.

[26] C. X. Ren, D. Q. Dai, and Y. Hong, "Robust classification using $\ell_{2,1}$-norm based regression model," *Pattern Recognit.*, vol. 45, no. 7, pp. 2708–2718, 2012.

[27] Y.-F. Yu, C.-X. Ren, M. Jiang, M.-Y. Sun, D.-Q. Dai, and G. Guo, "Sparse approximation to discriminant projection learning and application to image classification," *Pattern Recognit.*, vol. 96, Dec. 2019, Art. no. 106963.

[28] C. Li, X. Wang, W. Dong, J. Yan, Q. Liu, and H. Zha, "Joint active learning with feature selection via CUR matrix decomposition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 6, pp. 1382–1396, Jun. 2019.

[29] T. A. F. Gomes, R. B. C. Prudêncio, C. Soares, A. L. D. Rossi, and A. Carvalho, "Combining meta-learning and search techniques to select parameters for support vector machines," *Neurocomputing*, vol. 75, no. 1, pp. 3–13, 2012.

[30] C. Yang, Y. Akimoto, D. W. Kim, and M. Udell, "OBOE: Collaborative filtering for AutoML model selection," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2019, pp. 1173–1183.

[31] N. Fusi, R. Sheth, and M. Elibol, "Probabilistic matrix factorization for automated machine learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 3348–3357.

[32] S. Dyrmishi, R. Elshawi, and S. Sakr, "A decision support framework for AutoML systems: A meta-learning approach," in *Proc. Int. Conf. Data Mining Workshops (ICDMW)*, Nov. 2019, pp. 97–106.

[33] H. Bensusan and A. Kalousis, "Estimating the predictive accuracy of a classifier," in *Proc. Eur. Conf. Mach. Learn.* Berlin, Germany: Springer, 2001, pp. 25–36.

[34] S. B. Guerra, R. B. Prudêncio, and T. B. Ludermir, "Predicting the performance of learning algorithms using support vector machines as meta-regressors," in *Proc. Int. Conf. Artif. Neural Netw.* Berlin, Germany: Springer, 2008, pp. 523–532.

[35] A. C. Lorena, A. I. Maciel, P. B. C. de Miranda, I. G. Costa, and R. B. C. Prudêncio, "Data complexity meta-features for regression problems," *Mach. Learn.*, vol. 107, no. 1, pp. 209–246, Jan. 2018.

[36] K. A. Smith, F. Woo, V. Ciesielski, and R. Ibrahim, "Modelling the relationship between problem characteristics and data mining algorithm performance using neural networks," in *Proc. Smart Eng. Syst. Design: Neural Netw., Fuzzy Log., Evol. Program., Data Mining, Complex Syst.*, vol. 11, 2001, pp. 356–362.

[37] M. Kuck, S. F. Crone, and M. Freitag, "Meta-learning with neural networks and landmarking for forecasting model selection an empirical evaluation of different feature sets applied to industry data," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2016, pp. 1499–1506.

[38] K. A. Smith, F. Woo, V. Ciesielski, and R. Ibrahim, "Matching data mining algorithm suitability to data characteristics using a self-organizing map," in *Hybrid Information Systems*. Berlin, Germany: Springer, 2002, pp. 169–179.

[39] G. Obozinski, M. J. Wainwright, and M. I. Jordan, "Support union recovery in high-dimensional multivariate regression," *Ann. Statist.*, vol. 39, no. 1, pp. 1–47, 2011.

[40] M. Yuan and Y. Lin, "Model selection and estimation in regression with grouped variables," *J. Roy. Stat. Soc., B (Stat. Methodol.)*, vol. 68, no. 1, pp. 49–67, 2006.

[41] S. Noah, J. Friedman, T. Hastie, and R. Tibshirani, "A sparse-group lasso," *J. Comput. Graph. Stat.*, vol. 22, no. 2, pp. 231–245, 2013.

[42] Y. Li, B. Nan, and J. Zhu, "Multivariate sparse group lasso for the multivariate multiple linear regression with an arbitrary group structure," *Biometrics*, vol. 71, no. 2, pp. 354–363, Jun. 2015.

[43] X.-L. Xu, C.-X. Ren, R.-C. Wu, and H. Yan, "Sliced inverse regression with adaptive spectral sparsity for dimension reduction," *IEEE Trans. Cybern.*, vol. 47, no. 3, pp. 759–771, Apr. 2016.

[44] X.-F. Song, Y. Zhang, Y.-N. Guo, X.-Y. Sun, and Y.-L. Wang, "Variable-size cooperative coevolutionary particle swarm optimization for feature selection on high-dimensional data," *IEEE Trans. Evol. Comput.*, vol. 24, no. 5, pp. 882–895, Oct. 2020.

[45] X.-F. Song, Y. Zhang, D.-W. Gong, and X.-Z. Gao, "A fast hybrid feature selection based on correlation-guided clustering and particle swarm optimization for high-dimensional data," *IEEE Trans. Cybern.*, vol. 52, no. 9, pp. 9573–9586, Sep. 2022.

[46] E. Alcobaça, F. Siqueira, A. Rivolli, L. P. F. Garcia, J. T. Oliva, and A. C. P. L. F. de Carvalho, "MFE: Towards reproducible meta-feature extraction," *J. Mach. Learn. Res.*, vol. 21, no. 111, pp. 1–5, 2020.

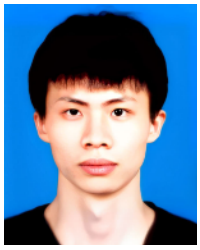
[47] E. R. Mansfield and B. P. Helms, "Detecting multicollinearity," *Amer. Statistician*, vol. 36, no. 3a, pp. 158–160, Aug. 1982.

[48] D. Dua and C. Graff, "UCI machine learning repository," School Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2019. [Online]. Available: https://archive.ics.uci.edu/ml/index.php

[49] C. Cortes and V. Vapnik, "Support-vector networks," *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, 1995.

[50] C.-W. Hsu et al., "A practical guide to support vector classification," Dept. Comput. Sci. Inf. Eng., Nat. Taiwan Univ., Taipei, Taiwan, Tech. Rep., 2003.

[51] A. I. Cowen-Rivers et al., "HEBO: Pushing the limits of sample-efficient hyper-parameter optimisation," *J. Artif. Intell. Res.*, vol. 74, pp. 1269–1349, Jul. 2022.



Liping Deng received the B.Sc. degree in applied mathematics from Anshan Normal University, Anshan, China, in 2016, and the M.S. degree in applied mathematics from Shenzhen University, Shenzhen, China, in 2019. He is currently pursuing the Ph.D. degree with the School of Mathematical and Statistical Science, Southern Illinois University Carbondale, Carbondale, IL, USA.

His research interests include machine learning and pattern recognition.

Wen-Sheng Chen received the B.S. and Ph.D. degrees in mathematics from Sun Yat-sen (Zhongshan) University, Guangzhou, China, in 1989 and 1998, respectively.

He joined the College of Science, Shenzhen University, Shenzhen, China, in 1999. From 2008 to 2009, he was a Visiting Professor with the Department of Computer Science, Hong Kong Baptist University, Hong Kong, China. He is currently a Professor with the College of Mathematics and Statistics and the Department of Information and Computational Science, Shenzhen University. His major research interests include image processing, pattern recognition, kernel-based machine learning, and wavelet analysis and its applications. He has published over 140 refereed scientific research articles in international journals and international conferences in these areas.

Mingqing Xiao received the Ph.D. degree in mathematics from the University of Illinois at Urbana–Champaign, Champaign, IL, USA, in 1997.

He is currently a Professor of mathematics with Southern Illinois University Carbondale, Carbondale, IL, USA. His current research interests include big data analysis, machine learning, optimization theory, and computational science.

Dr. Xiao is on the many editorial boards of various journals, such as IEEE TRANSACTIONS ON AUTOMATIC CONTROL, *Automatica*, and *European Journal of Control*.

Authorized licensed use limited to: Universidade Federal de Alagoas. Downloaded on May 06,2026 at 12:56:00 UTC from IEEE Xplore. Restrictions apply.